Applied NLP

DATA 641

Lecture 5 — Deep Learning

*Figures and notes are from the books Deep Learning with Python, Second Edition by François Chollet, Manning, ISBN 9781617296864 and Hands-On Machine Learning with Scikit-Learn Keras and TensorFlow, Aurelien Geron, O'Reilly

Quick history and facts

- Artificial neural networks (ANNs) were first introduced back in 1943 by the neurophysiologist Warren McCulloch and the mathematician Walter Pitts. In their landmark paper,2 "A Logical Calculus of Ideas Immanent in Nervous Activity," McCulloch and Pitts presented a simplified computational model of how biological neurons might work together in animal brains to perform complex computations using propositional logic.

- In the early 1980s there was a revival of interest in connectionism (the study of neural networks), as new architectures were invented and better training techniques were developed. But progress was slow!

- By the 1990s other powerful Machine Learning techniques were invented, such as Support Vector Machines. These techniques seemed to offer better results and stronger theoretical foundations than ANNs, so once again the study of neural networks entered a long winter.

- Geoffrey Hinton + Yoshua Bengio + Yann Lecun received the **2018 Turing Award** for their contributions to the field of neural networks.

Quick history and facts (cont.)

- Finally, we are now witnessing yet another wave of interest in ANNs! Why?
  - There is now a huge quantity of data available to train neural networks, and ANNs frequently outperform other ML techniques on very large and complex problems.
  - The tremendous increase in computing power since the 1990s now makes it possible to train large neural networks in a reasonable amount of time. This is in part due to Moore's Law, but also thanks to the gaming industry, which has produced powerful GPU cards by the millions.
  - The training algorithms have been improved.
  - ANNs seem to have entered a virtuous circle of funding and progress. Amazing products based on ANNs regularly make the headline news, which pulls more and more attention and funding toward them, resulting in more and more progress, and even more amazing products.

A first look at a neural network

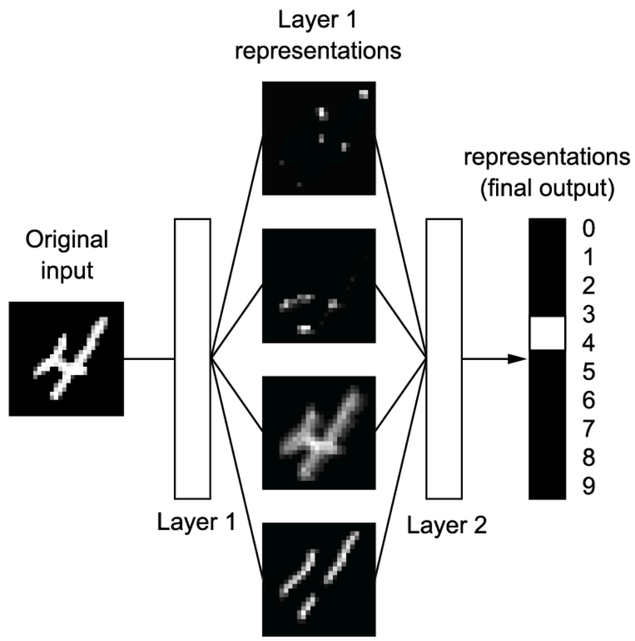
The problem we are trying to solve here is to classify gray-scale images of handwritten digits (28 × 28 pixels) into their 10 categories (0 through 9).

```python
from tensorflow.keras.datasets import mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

print(train_images.shape)

print(len(train_labels))

print(train_labels)

print(test_images.shape)

print(len(test_labels))

print(test_labels)
```

```
(60000, 28, 28)
60000
[5 0 4 ... 5 6 8]
(10000, 28, 28)
10000
[7 2 1 ... 4 5 6]
```

The network architecture

The core building block of neural networks is the **layer**: Layers extract representations out of the data fed into them



- Model consists of a sequence of two `Dense` layers which are densely connected (also called fully connected) neural layers
- Second (and last) layer is a 10-way **softmax classification** layer, which means it will return an array of 10 probability scores (summing to 1)

```
from tensorflow import keras
from tensorflow.keras import layers
model = keras.Sequential([
    layers.Dense(512, activation="relu"),
    layers.Dense(10, activation="softmax")
])
```

To make the model ready for training, we need to pick three more things as part of the **compilation** step:

- An **optimizer**: The mechanism through which the model will update itself based on the training data it sees, so as to improve its performance.
- A **loss function**: How the model will be able to measure its performance on the training data, and thus how it will be able to steer itself in the right direction.
- **Metrics to monitor during training and testing**: Here, we will only care about accuracy (the fraction of the images that were correctly classified).

```
# The compilation step

model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

Preparing the image data

- Pre-process the data by reshaping it into the shape the model expects and scaling it so that all values are in the $[0, 1]$ interval.
- Originally, our training images were stored in an array of shape `(60000, 28, 28)` of type `uint8` with values in the $[0, 255]$ interval.
- Transform it into a `float32` array of shape `(60000, 28 * 28)` with values between 0 and 1.

```
train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype("float32") / 255
test_images = test_images.reshape((10000, 28 * 28))
test_images = test_images.astype("float32") / 255
```

Fit the model to its training data

Here there are two new terms that need to be defined

- The batch size is a number of samples processed before the model is updated
- The number of epochs is the number of complete passes through the training dataset

```
model.fit(train_images, train_labels, epochs=5, batch_size=128)
```

```
Epoch 1/5
469/469 [==============================] - 3s 3ms/step - loss: 0.2522 - accurac
y: 0.9274
Epoch 2/5
469/469 [==============================] - 1s 3ms/step - loss: 0.1036 - accurac
y: 0.9692
Epoch 3/5
469/469 [==============================] - 1s 3ms/step - loss: 0.0683 - accurac
y: 0.9796
Epoch 4/5
469/469 [==============================] - 1s 3ms/step - loss: 0.0497 - accurac
y: 0.9851
Epoch 5/5
469/469 [==============================] - 1s 3ms/step - loss: 0.0375 - accurac
y: 0.9888

<keras.callbacks.History at 0x1f70106c220>
```

Two quantities are displayed during training:

- The loss of the model over the training data;
- The accuracy of the model over the training data.

Using the model to make predictions

```
test_digits = test_images[0:10]
predictions = model.predict(test_digits)

print(predictions[0])
print(predictions[0].argmax())
print(predictions[0][7])
print(test_labels[0])
```

```
1/1 [==============================] - 0s 56ms/step
[1.9335996e-08 8.4500390e-10 4.8529123e-06 6.2377658e-05 2.3709999e-11
 6.2441621e-09 3.0558685e-14 9.9992740e-01 8.0540850e-08 5.2332321e-06]
7
0.9999274
7
```

Evaluating the model on new data

```
test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f"test_acc: {test_acc}")
```

```
313/313 [==============================] - 1s 3ms/step - loss: 0.0648 - accurac
y: 0.9810
test_acc: 0.9810000061988831
```